

Machine Learning para Clasificar Usuarios de Aplicación Musical

JUAN SEBASTIÁN CHITIVA GUERRERO, Tecnológico de Monterrey, MX

Se utiliza un dataset de información sobre una app llamada "Playlist Pro" con el propósito de analizar el uso y características principales de usuarios y así encontrar patrones para clasificar de manera binaria a los usuarios activos y los que han cancelado su suscripción. Este análisis contiene una auditoría de datos, la codificación de variables categóricas usando One Hot Encoding, estandarización de variables numéricas y análisis exploratorio de correlaciones. La clasificación se realiza inicialmente con un modelo de regresión logística sin uso de frameworks enfocados en Machine Learning y posteriormente se implementan mejoras con el propósito de encontrar el resultado más aproximado y óptimo al problema. Como métricas de evaluación se utiliza Accuracy, Recall de la clase positiva y la matriz de confusión.

ACM Reference Format:

Juan Sebastián Chitiva Guerrero. 2018. Machine Learning para Clasificar Usuarios de Aplicación Musical. *ACM Trans. Graph.* 37, 4, Article 111 (August 2018), 5 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

La música es un rasgo relevante y muy importante para la vida de muchas personas alrededor del mundo. Es su forma de despejarse y de relajar la mente. Gracias a esto y con el avance de la tecnología, existen actualmente muchas aplicaciones para poder escuchar música. En este reporte, se busca analizar las características y formas de uso de los clientes para determinar que provoca a los usuarios cancelar la suscripción de la aplicación "Playlist Pro". Este análisis se realiza para poder implementar medidas que minimicen los casos de cancelación.

2 Comprensión de los datos

2.1 Información Inicial

Se ha hecho el análisis con un dataset que contiene 41,000 muestras y 20 columnas. Cada muestra es un usuario y las columnas determinan información característica sobre el usuario y el uso de la aplicación, métodos de pago, fecha de suscripción, entre otros. El dataset se puede encontrar en la siguiente página. https://www.kaggle.com/daliado98/music-streaming-customer-churn-dataset/data?select=sample_submission.csv

2.2 Objetivos Analíticos

Se busca construir un modelo de clasificación que permita determinar a los usuarios activos contra los clientes que han cancelado su suscripción. Este resultado será arrojado de forma binaria, en el cual el valor "1" determinar al usuario que ha cancelado.

Author's Contact Information: Juan Sebastián Chitiva Guerrero, a01276239@tec.mx, Tecnológico de Monterrey, Querétaro, Querétaro, MX.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7368/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

2.3 Definiciones y métricas clave

Definiciones:

- **Churned:** Persona que ha cancelado su suscripción (0 = activo, 1 = cancelado)

Métricas clave:

Para poder determinar la efectividad de los modelos, se buscará usar las siguientes métricas:

- **Accuracy:** Este determina la efectividad en predecir los usuarios.
- **Recall de clase positiva:** Determina la efectividad en predecir a los usuarios que si han cancelado su suscripción.

2.4 Análisis inicial de Dataset

La base de datos contiene 41,000 muestras y los siguientes atributos:

Table 1. Diccionario de variables

Variable	Descripción	Tipo
Customer ID	Identificador de cada cliente	Númerica
Age	Edad del cliente	Númerica
Location	Estado de residencia del cliente	Categórica
Subscription Type	Tipo de suscripción	Categórica
Payment Plan	Forma de pago de suscripción: anual o mensual	Categórica
Num Subscription Pauses	Número de pausas de la suscripción (máx. 2)	Categórica
Payment Method	Forma de pago del usuario	Categórica
Customer Service Inquiries	Frecuencia de uso del servicio a cliente	Categórica
Sign up Date	Fecha en la que comenzó la suscripción	Fecha
Weekly Hours	Promedio mensual de horas que escucha música	Númerica
Average Session Length	Promedio de duración de cada sesión	Númerica
Song Skip Rate	Porcentaje de canciones que el usuario salta	Númerica
Weekly Songs Played	Promedio de canciones escuchadas por semana	Númerica
Unique Songs	Promedio de canciones únicas escuchadas por semana	Númerica
Favorite Artists	Número de artistas definidos como favoritos	Númerica
Platform	Número de conexiones con otros usuarios	Númerica
Friends	Número de playlists creadas	Númerica
Num Playlists Created	Número de playlists compartidas públicamente	Númerica
Shared Playlists	Número de notificaciones en las que hizo clic	Númerica
Notifications Clicked	Indica si el usuario está activo o se dio de baja	Categórica de clase
Churned		

3 Auditoría de Datos

Se identifican las variables con valores nulos o vacíos, los cuales al analizar el Dataset, no se encontraron ningunos.

También se realizó la identificación de errores en variables categóricas con el objetivo de encontrar errores ortográficos, gramaticales o de espacios que pudieran afectar los resultados. Se encontró un error en la variable "Location", en la cual se describía un estado como "Nebrasksa". Se ha sustituido a "Nebraska".

3.1 Codificación de Variables Categóricas

En las variables categóricas definidas en [1], se realiza una codificación a valores numéricos de tal manera que puedan ser incluidos como variables predictoras en los modelos. Este proceso se realiza con el método de One Hot Encoding, el cual busca convertir cada categoría en un resultado binario.

3.2 Reestructuración de Columnas

Se define el Index por la columna "customer_id" para tratar de darle mayor limpieza y estructura a la base de datos.

3.3 Estandarización de variables numéricas

Ya que existe diferencia en unidades para las variables numéricas, se ha determinado su estandarización. Esto se ha realizado con el método de Standard Scaler, el cual busca ajustar los valores de cada columna utilizando su desviación y su media. Con esto se logra tener las variables en una escala que permita que sean comparables entre ellas.

4 Análisis Exploratorio

4.1 Correlaciones

Para encontrar posibles relaciones entre variables numéricas, se ha utilizado una matriz de correlación que nos permita tener una observación inicial de si existen comportamientos proporcionales entre las variables.

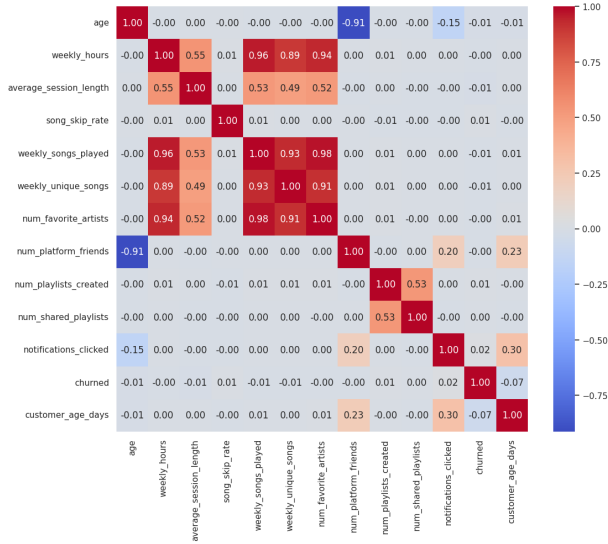


Fig. 1. Matriz de Correlación entre Variables Numéricas.

Observando la matriz de correlación, se puede interpretar que existe una relación muy alta entre las columnas "weekly_hours", "weekly_songs", "weekly_unique_songs" y "num_favorite_artists". Esto tiene sentido debido a que al tener una mayor cantidad de música escuchada, aumenta la cantidad de canciones.

También se puede observar que la relación entre el número de artistas favoritos con el número de horas se podría interpretar como

el grupo de usuarios que usan frecuentemente la aplicación y tienen una actividad alta en esta.

Analizando esta información y para reducir información que pueda ser redundante en el modelo, se busca eliminar las variables altamente correlacionadas y solo mantener el atributo de "weekly_hours".

4.2 Datos Relevantes

Analizando la columna predictora "churned", contiene 32,253 usuarios activos y 8,747 que han cancelado su suscripción. Esto en porcentaje se interpreta como 78.7% y 21.3% respectivamente.

5 Selección y Preparación de Modelo

5.1 Selección de Modelo

En el dataset, se busca predecir la variable "churned" que está clasificada de forma binaria. Al buscar un modelo de aprendizaje supervisado que permita predecir de forma correcta esta variable, se determinó el usar el modelo de **regresión logística**, el cual permite estimar la probabilidad de a qué clase pertenece el usuario en base a sus características y usos en la aplicación con el propósito de encontrar relaciones entre estos e identificar posibles usuarios que busquen cancelar su suscripción y así implementar nuevas estrategias para minimizar usuarios cancelados.

5.2 Funciones de Modelo

Para el modelo, se utiliza la función sigmoideal, la cual busca transformar el resultado de la combinación lineal de las variables independientes en un valor que está en un intervalo de 0 y 1. de esta forma, se puede interpretar como probabilidad de pertenecer entre clases. En este caso, se buscaría utilizar las características de la aplicación por usuario y predecir la pertenencia de cada uno.

Esta función tiene como entrada la combinación lineal de las variables y los pesos. Se define de la siguiente manera

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n. \quad (1)$$

Con el resultado de esta ecuación, se define la función sigmoideal como:

$$y = \frac{1}{1 + e^{-z}}. \quad (2)$$

El cual, como se menciona anteriormente, busca representar cada resultado como una probabilidad de pertenecer en la clase 1.

5.3 Selección de Datos

Una vez realizado el análisis exploratorio de datos y con la elección del modelo de regresión logística, se busca seleccionar las variables independientes que serán usadas para entrenar al modelo. Se define la columna "**churned**" como la variable a predecir (dependiente). Las variables dependientes serán las siguientes:

- Age
- Location
- Subscription Type
- Payment Plan
- Average Session Length
- Num Subscription Pauses
- Payment Method

- Customer Service Inquiries
- Sign up Date
- Weekly Hours
- Num Platform Friends
- Num Playlists Created
- Num Shared Playlists
- Notificaciones Clicked

Cabe mencionar que las variables categóricas ya han sido transformadas de forma binaria a través de One Hot Encoding.

5.4 Split de Dataset

Tras agrupar las columnas para el entrenamiento del modelo, se busca dividir la información en dos conjuntos de datos: Test y Train. La parte de Test es la que contiene la mayor cantidad de información de los datos, el cual se usa para entrenar el modelo. Test se usa para probar el modelo y mostrar su eficacia. En este caso, se ha utilizado un 70% del dataset en entrenar el modelo y un 30% para hacer el test.

6 Entrenamiento del modelo

Antes de iniciar el entrenamiento, se ha incorporado un bias antes de cada observación, de tal manera que pueda formar parte de la combinación lineal que se utiliza en la función de hipótesis.

Sabiendo esto, el modelo comienza inicializando los pesos respecto a las variables dependientes. Para cada observación se calcula una combinación lineal entre sus características y sus pesos. Una vez hecho esto, se ingresa a la función sigmoideal para obtener su probabilidad de pertenencia hacia la clase 1.

Cuando termina de realizar esto, se comparan los resultados de la predicción respecto al valor real de la variable objetivo y la diferencia entre estos determina el error de la predicción.

En base a este error, se ajustan los pesos con el método de gradiente descendiente, el cual busca ajustar los pesos de tal manera que minimice el error del modelo lo más posible. En cada iteración busca calcular el gradiente de las observaciones y se actualizan con una tasa de aprendizaje (α) que ya es definida anteriormente.

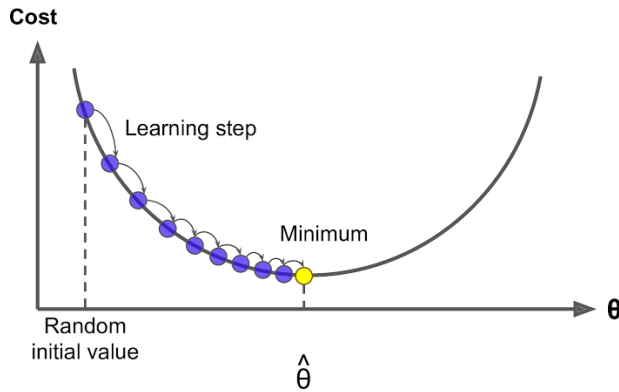


Fig. 2. Representación visual del gradiente descendiente

Una vez implementada esta fórmula, los pesos se irán actualizando a través de las iteraciones de forma sucesiva en la dirección que haga que el costo disminuya.

Después de que se actualizan los pesos, se calculan nuevas probabilidades de cada muestra con los nuevos pesos y así se determina el costo del modelo utilizando la función de entropía cruzada (*Cross Entropy*).

$$\text{cost} = -\frac{1}{M} \sum_{i=1}^M [y_i \log(h_0(x_i)) + (1 - y_i) \log(1 - h_0(x_i))] \quad (3)$$

Esta función de costo permite medir la distancia entre las probabilidades estimadas por el modelo respecto a las clases reales. El menor valor de costo representa un mejor ajuste entre el modelo y las clases reales.

Cuando termina la primera iteración, se almacena el valor de costo que ha arrojado la entropía cruzada. Esto se utiliza al final de entrenar el modelo para observar el comportamiento durante las iteraciones y como va mejorando mientras avanzan estos.

En el modelo se establece un máximo de 100 iteraciones. Este límite se utiliza para que el modelo no siga por iteraciones extremadamente largas ajustando los pesos buscando el óptimo. Puede llegar a tener mucho costo computacional o incluso ser redundante si en pocas iteraciones permite converger a una solución adecuada. El modelo puede terminar sus iteraciones de dos maneras.

- (1) El modelo llegó al límite de iteraciones.
- (2) El modelo alcanza una condición de convergencia definida.

Se acostumbra tener una mayor cantidad de iteraciones, pero al ser un modelo sin framework y que se considera "rústico", toma un tiempo considerable en realizar cada iteración.

7 Resultados

Para analizar los resultados, se incluye la gráfica de costo a través de cada iteración.

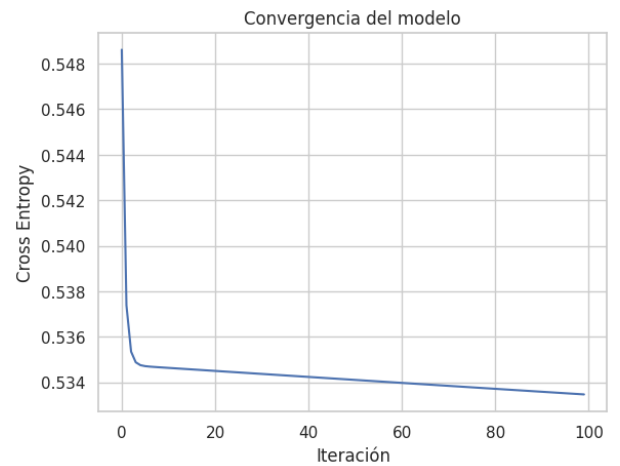


Fig. 3. Convergencia del modelo a través de las iteraciones.

En el gráfico se observa que, en el transcurso de las iteraciones, se va disminuyendo el costo del modelo. Por lo tanto, si está funcionando de una manera adecuada el modelo. Cabe recordar que no es un modelo óptimo y que no contiene una cantidad grande de iteraciones.

Para las métricas, se ha utilizado Accuracy, Recall de la clase 1 y la matriz de confusión. Accuracy nos permite saber qué tantas predicciones fueron correctas respecto a toda la base de predicción. Es decir, de todo el set de prueba, cuántos logró predecir de forma adecuada el modelo.

Recall sobre la clase 1 permite saber cuántos usuarios que realmente cancelaron se lograron detectar en base a las predicciones del modelo. Ambas métricas se obtienen a través de la matriz de confusión, la cual nos permite analizar la predicción de cada una de las clases. Esto lo hace a través de falsos y positivos. Captura cuáles sí fueron predichos de forma adecuada y cuáles fueron falsos positivos y falsos negativos.

Los resultados del modelo son los siguientes:

Table 2. Resultados del modelo inicial

Métrica	Valor
Probabilidad mínima	0.1225
Probabilidad máxima	0.3982
Probabilidad promedio	0.2416
True Positive	0
True Negative	9685
False Positive	0
False Negative	2615
Recall clase 1	0.0
Accuracy	0.7874

Analizando los resultados que ha arrojado el set de prueba (Test), se observa que el modelo tiene un accuracy de 78%, lo cual no está mal. Pero, al analizar el recall de la clase 1 indica que es de 0%.

Cabe mencionar que el umbral de decisión tras implementar la función sigmoidal es de 0.5. Es decir, si mi probabilidad de churn es mayor a 0.5, se le asigna la clase 1. Observando la probabilidad máxima, se determina que es casi un 40% de probabilidad, esto da a entender que el modelo predice que ningún usuario llega a cruzar ese umbral de 50%. Es por eso que el recall es de 0%, no existe ningún usuario que haya sido clasificado dentro de la clase 1.

Es importante entender esto y observar la importancia de tener múltiples métricas de evaluación para el modelo. Aunque el modelo muestra un accuracy relativamente alto, se muestra que no cumple lo que se busca ya que todos los predice como clase 0. Esto es lo que sesga a que haya un accuracy alto, ya que la mayoría de las muestras del dataset son de clase 0.

7.1 Interpretación del modelo inicial

El modelo inicial al ser una regresión logística simple creada sin ningún framework, limita la cantidad de iteraciones y posiblemente la efectividad de encontrar una relación entre las variables para clasificar entre los usuarios. Se buscaría implementar un modelo

que sea más robusto que tenga una mayor fuerza y capacidad para poder encontrarlos.

8 Implementación de Regresión Logística usando Scikit-Learn

Tras correr el modelo de forma manual e interpretar sus resultados, se buscaría observar el comportamiento y predicción al usar Scikit-Learn para realizar una Regresión Logística con distintos parámetros.

Se utilizaría el mismo set de datos, la única diferencia es que se implementaría el split de datos que viene incluido en Scikit Learn, así como el Standard Scaler y el modelo de Regresión Logística.

Para implementar en el framework, se buscaría hacer un split de datos que incluya un 63% de los datos en Train, un 7% de validation y un 30% de test. Se buscaría utilizar el comando de “stratify=y_train” ya que este permite que se tenga la misma proporción de clases entre el set de train y validation.

Una vez implementado el Scale de datos, se correría el nuevo modelo con los siguientes hiperparámetros:

- Solver = lbfgs
- max_iterations = 300
- Penalty = None

El hiper parámetro define el método de optimización del modelo, el cual es “Limited-memory Broyden–Fletcher–Goldfarb–Shanno”. Este es un optimizador de segundo orden que busca aproximar las derivadas de segundo orden utilizando las derivadas de pocas iteraciones anteriores. Es el optimizador por default en el framework de scikit learn. El número de iteraciones lo limitamos a 300 para que tenga mayor tiempo para converger hacia una solución y no se limite como el modelo que se realizó a mano. Penalty busca eliminar la regularización del modelo, la cual ayuda a castigar los modelos que se ajustan a los sets de entrenamiento y no se comportan de una forma independiente ya que “memorizó” lo que se usó en su proceso de entrenamiento. En otras palabras, busca eliminar el “overfitting” del modelo. Este puede que sea utilizado posteriormente en caso de que lo requiera, pero por el momento no será utilizado.

8.1 Resultados Modelo con Framework

Tras correr el modelo, se observan las métricas para así entender los resultados y poder determinar si es adecuado o si necesita ajustes. Se ha buscado obtener métricas de cada parte del dataset, los cuales son las siguientes,

8.1.1 Train.

- **Accuracy:** 0.7862%
- **Recall:** 0.0014%

Matriz de confusión:

Table 3. Matriz de confusión para el conjunto de entrenamiento.

	Predicción 0	Predicción 1
Real 0	20,299	6
Real 1	5,517	8

8.1.2 Validation.

- **Accuracy:** 0.7857%
- **Recall:** 0.0000%

Matriz de confusión:

Table 4. Matriz de confusión para el conjunto de validación.

	Predicción 0	Predicción 1
Real 0	2,255	1
Real 1	614	0

8.1.3 Test.

- **Accuracy:** 0.7878%
- **Recall:** 0.0038%

Matriz de confusión:

Table 5. Matriz de confusión para el conjunto de prueba.

	Predicción 0	Predicción 1
Real 0	9,680	12
Real 1	2,598	10

Así como el modelo de regresión logística hecho a mano, se tiene un accuracy de aproximadamente 78%, el cual ya hemos declarado ser engañoso. También es importante mencionar que en todos los casos la métrica de “Recall” es prácticamente 0. Se buscará analizar principalmente los resultados de validation [4].

Al observar su matriz de confusión, se interpreta que el modelo predice de manera correcta la gran mayoría de los usuarios que son activos, a excepción de uno. Ahora, en el caso de los usuarios que queremos encontrar, no ha sido posible determinar ni una sola vez de manera adecuada. Generalizando el comportamiento y desempeño del modelo, se observa que se está sesgando fuertemente hacia la clase 0 y no está identificando de manera adecuada a los usuarios que entran en la clase 1. Un efecto de esto puede ser el desbalance de clase, ya que, como se mencionó en la exploración de datos, existen 78.8% de usuarios en clase 0. No significa que este sea el principal problema, solo se está identificando el desbalance y al mismo tiempo no está encontrando una relación significativa entre los features que haga que clasifique de forma directa o “sencilla” entre ambas clases. Adentrándose más en la información que se obtuvo del modelo, se busca encontrar como clasificó las probabilidades de cada una de las clases y como están distribuidas. Se grafica una caja de bigotes para cada una, mostrando las probabilidades de que pertenezcan a la clase 1, así como su threshold que es el que determina clasificar uno u otro.

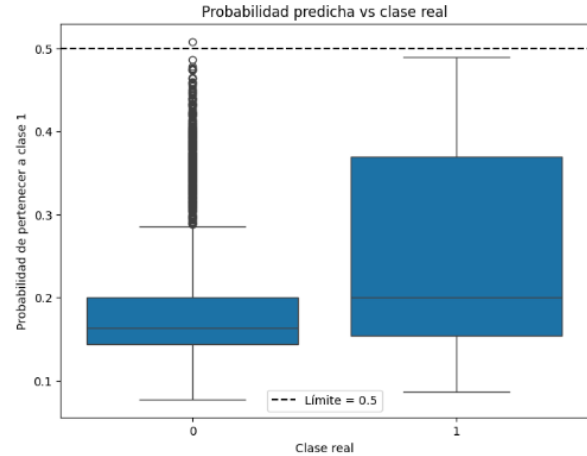


Fig. 4. Caja de Bigotes de probabilidad por clase.

Se puede observar que la caja de bigotes de la clase 0 se concentra su probabilidad en un rango de 0.15% y 0.2%. Esto da a entender que el modelo si logra concentrar por lo menos un 50% de los datos en un rango relativamente pequeño. Con esto, se observa que si existe una gran cantidad de usuarios que la caja de bigotes considera como outliers. En el caso de la clase 1, se observa que si es mucho más disperso y su mediana también se concentra en el 20% de probabilidad. Esto ayuda entender más la información, pero también demuestra que el modelo no está encontrando una relación que permita separar de forma clara a los usuarios activos y los que han cancelado su suscripción.

9 Mejoras

References

- [1] E. Raza, “Gradient Descent,” *Medium*, Sep. 5, 2019. [Online]. Available: <https://medium.com/@raza.shan83/gradient-descent-c568801d0b62>
- [2] Scikit-learn developers, “LogisticRegression,” *Scikit-learn Documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- [3] Novus AFK, “Understanding Logistic Regression: A Beginner’s Guide,” *Medium*. [Online]. Available: https://medium.com/@novus_afk/understanding-logistic-regression-a-beginners-guide-73f148866910